

Control Statements.

Control statements in Java determine the execution flow of your program. Without them, code simply runs line-by-line from top to bottom.

Java divides control statements into three main categories: **Selection** (Decision-making), **Iteration** (Loops), and **Jump** statements.

1. Selection Statements (Decision Making)

These statements evaluate a boolean condition to decide which block of code to run.

Decision Making statements	Java code
if Statement Executes a block of code only if its condition is true. [	<pre>int age = 20; if (age >= 18) { System.out.println("You can vote."); }</pre>
if-else Statement Provides an alternative block of code if the condition is false.	<pre>int score = 65; if (score >= 70) { System.out.println("Passed"); } else { System.out.println("Failed"); }</pre>
if-else-if Ladder Tests multiple conditions in sequence. It stops at the first true condition.	<pre>int speed = 85; if (speed > 90) { System.out.println("Heavy fine!"); } else if (speed > 70) { System.out.println("Warning ticket."); } else { System.out.println("Safe driving."); }</pre>
switch Statement Compares a single variable against multiple specific constant values (cases). It is cleaner than a long if-else ladder when checking one variable.	<pre>int day = 2; switch (day) { case 1: System.out.println("Monday"); break; case 2: System.out.println("Tuesday"); break; // Matches and prints default: System.out.println("Invalid day"); }</pre>

2. Iteration Statements (Loops)

Loops repeat a block of code as long as a specific condition remains true.

Iteration Statements	Java code
while Loop Checks the condition before executing the loop body. If the condition is false initially, the loop never runs.	<pre>int count = 1; while (count <= 3) { System.out.println(count); count++; // Increments count so the loop eventually ends }</pre>
do-while Loop Executes the loop body first , then checks the condition. This guarantees the code runs at least once .	<pre>int x = 10; do { System.out.println("This runs once even though x > 5"); } while (x < 5);</pre>
for Loop Combines initialization, condition, and increment/decrement into one line. Best used when you know exactly how many times to repeat.	<pre>for (int i = 0; i < 3; i++) { System.out.println("Iteration: " + i); }</pre>
For-Each Loop (Enhanced for) Specifically designed to iterate through all elements in an array or collection without using counter indexes.	<pre>int[] numbers = {10, 20, 30}; for (int num : numbers) { System.out.println(num); }</pre>

3. Jump Statements

Jump statements instantly transfer the program's control to a different part of the code.

Jump Statements	Java code
break Immediately terminates the innermost loop or switch statement.	<pre>for (int i = 1; i <= 5; i++) { if (i == 3) { break; // Stops the loop completely when i is 3 } System.out.println(i); // Prints 1, 2 }</pre>
continue Skips the remaining code inside the loop for the current iteration and jumps directly to the next turn of the loop.	<pre>for (int i = 1; i <= 4; i++) { if (i == 2) { continue; // Skips printing 2 } System.out.println(i); // Prints 1, 3, 4 }</pre>
return Explicitly returns from a method. It stops the execution of the entire method and passes control back to where the method was called.	<pre>public int doubleValue(int number) { return number * 2; // Exits method and sends value back }</pre>